

Part A:

Cycle 1:

I_1 issued to Load₁. ~~No depend~~ $RST[F_2] = \text{null}$, nothing to wait for.

RS	Busy	Op	V_j	V_k	Q_j	Q_k
Load ₁	Yes	LD	$[R_1]$	-	-	-

RST: All null $F_2 \rightarrow \text{Load}_1$

CDB: Nothing

Cycle 2:

I_2 issued to Multi₁. ~~but~~ $RST[F_2] = \text{Load}_1$ (wait)

$F_2 \rightarrow$ free for use.

RS	Busy	Op	V_j	V_k	Q_j	Q_k
Load ₁	Yes (Ex)	LD	$[R_1]$	-	-	-
Multi ₁	Yes	MUL	-	$[F_2]$	Load ₁	-

RST: $F_2 \rightarrow \text{Load}_1$ $F_4 \rightarrow \text{Multi}_1$

CDB: (~~Load₁, mem~~) Nothing

Cycle 3:

I_3 issued to Add₁. ~~RST~~ Needs F_2 .

I_1 done. $\Rightarrow RST[F_2] = \text{Null}$, (Load₁, Mem $[R_1]$) on CDB

RS	Busy	Op	V_j	V_k	Q_j	Q_k
Load ₁	No					
Multi ₁	Yes (ready)	MUL	$[F_2]$	$[F_2]$	-	-
Add ₁	Yes (ready)	ADD	$[F_2]$	$[F_0]$	-	-

$\hookrightarrow$ captured and loaded onto F_2 .

$\bullet$ Taken to Multi₁, Ad

$RST[F_2] = \text{null}$.

RST: $F_2 \rightarrow \text{Null}$, $F_4 \rightarrow \text{Multi}_1$, $F_8 \rightarrow \text{Add}_1$.

CDB: (Load₁, mem $[R_1]$)

Cycle 4:

I₄ issued to Add₂. RST[F₄] = ~~sub~~ Add₂. waits on F₈.

Rs	Busy	Op	V _j	V _k	Q _j	Q _k
Load ₁	No					
Mult ₁	Yes (Ex)	MUL	[F ₂]	[F ₆]	-	-
Add ₁	Yes (Ex)	ADD	[F ₂]	[F ₁₀]	-	-
Add ₂	Yes	SUB	-	[F ₂]	Add ₁	-

RST: F₄ → ~~Add₂~~, F₈ → Add₁

CDB: Nothing

Cycle 5:

I₅ issued to Load₁ RST[F₈] = Load₁

I₃ done. CDB: (Add₁, result₁) → picked up by Add₂. ~~not~~ loaded in F₈.

Rs	Busy	Op	V _j	V _k	Q _j	Q _k
Mult ₁	Yes (Ex)	MUL	[F ₂]	[F ₆]	-	-
Add ₁	No					
Add ₂	Yes (ready)	SUB	[F ₈]	[F ₂]	-	-
Load ₁	Yes	LD	[R ₁]			

RST: F₄ → Add₂, F₈ → Load₁.

CDB: (Add₁, result₁).

Cycle 6:

I₆ issued to Add₁ RST[F₁₂] = Add₁. waits on F₄ and F₈

RS	Busy	Op	V _j	V _k	Q _j	Q _k
Mult ₁	Yes (Ex)	MUL	[F ₂]	[F ₆]	-	-
Add ₂	Yes (Ex)	SUB	[F ₈]	[F ₂]	-	-
Load ₁	Yes (Ex)	LD	[R ₁]			
Add ₁	Yes	ADD	-	-	Add ₂	Load ₁

RST: F₄ → Add₂, F₈ → Load₁, F₁₂ → Add₁

Cycle 7:

Both Add2, Load1 complete and broadcast. Add1 is ready.

RST	Busy	OP	V _j	V _k	Q _j	Q _k
Multi	Yes (Ex)	MUL	[F2]	[F6]	-	-
Add2	No					
Load1	No					
Add1	Yes (Ready)	ADD	[F4]	[F8]	-	-

RST: F12 → Add1

CDB: (Add2, result2), (Load1, Mem[R1+8])

Cycle 8: → Execution only

Cycle 9: → All done

Multi, Add1 Finish.

RST	Busy	OP	V _j	V _k	Q _j	Q _k
Multi	No					
Add2	No					
Load1	No					
Add1	No					

RST: All Null

CDB: (Add1, result3), (Multi, result4)

Part - B:

Taking I_2, I_4 from Part-A;

Quite Obviously, any instruction between I_2 & I_4 must use result from I_2 which takes 6 cycle to be done (at least) and any instruction after I_4 must take value of I_4 if needed.

Thus, ~~we~~ all we do is point them to I_2 or I_4 accordingly instead of being hung up on the register itself. In a way it is like drinking water straight from the jug (RS) instead of waiting for a glass to be filled (Register update), a mixup occurs when a simple minded waiter^(CPU) pours wine into glass ~~and~~ due to a following instruction heard when fetching the water.

Anyways, to ensure fresh info reaches instructions RST constantly points to most recent update. Even when I_2 finishes it does not write back to register since the most recent info (I_4) is already there.